make sense. Using placeholders instead of interpolated strings gives you the same output message but also creates key-value pairs that can be searched later.

We've looked a lot at how you can create log messages in your app, but we haven't focused on where those logs are written. In the next section we'll look at the built-in ASP.NET Core logging providers, how they're configured, and how you can add a third-party provider.

26.3 Controlling where logs are written using logging providers

In this section you'll learn how to control where your log messages are written by adding `ILoggerProvider`s to your application. As an example, you'll see how to add a simple file logger provider that writes your log messages to a file, in addition to the existing console logger provider.

Up to this point, we've been writing all our log messages to the console. If you've run any ASP.NET Core sample apps locally, you'll probably have seen the log messages written to the console window.

NOTE If you're using Visual Studio and debugging by using the *Internet Information* Services (IIS) Express option, you won't see the console window (though the log messages are written to the Debug Output window instead).

Writing log messages to the console is great when you're debugging, but it's not much use for production. No one's going to be monitoring a console window on a server, and the logs wouldn't be saved anywhere or be